

SAFECode Fundamental Practices for Secure Software Development — an overview

Tiago Oliveira
tiago@di.uminho.pt

1. What is (and is not) **SAFECode** Fundamental Practices for Secure Software Development

- ▶ **SAFECode** presents a **foundational** set of secure development practices used in real organizations.
- ▶ The document is meant to help teams **start or improve** a secure development lifecycle (SDL) program.
- ▶ It is explicitly **not a complete or exhaustive guide**. It gives a core baseline, not the final word on software security.
- ▶ The paper emphasizes **“practiced practices”**: methods that were actually used, not just theoretical recommendations.

Note: Read the paper as a practical map of an Secure Software Development Lifecycle, not as a complete textbook of security.

Source: **SAFECode**, Fundamental Practices for Secure Software Development (2018), pp. 4-6

2. Why SAFECODE wrote this edition

- ▶ The third edition updates the older guidance because the threat landscape, tooling, and software delivery models changed. Since this document is from 2018, keep an eye out for updates.
- ▶ The document reacts to:
 - faster and more iterative release cycles,
 - increased use of cloud and online services,
 - heavier dependence on third-party components,
 - growing compliance pressure.
- ▶ The document argues that these changes do **not** remove the need for discipline; they make discipline more important.

Note: Modern delivery speed is not an excuse to postpone security; it changes how security must be integrated.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 4-6

3. High-level structure of the document

1. Application security controls (security requirements)
 2. Design
 3. Secure coding practices
 4. Manage security risk w.r.t. the use of third-party components
 5. Testing and validation
 6. Managing security findings
 7. Vulnerability response and disclosure
 8. Planning the implementation of an SDL
- The order matters: the document moves from **prevention**, to **detection**, to **response**, to **program management**.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 2-3

4. Main idea: security is a lifecycle property

- ▶ Software assurance is not achieved by one tool, one review, one checklist, or one heroic expert.
- ▶ Security must be addressed **throughout the product lifecycle**: planning, design, coding, testing, release, and post-release response.
- ▶ The SDL should be **firm in security goals** but **flexible in application**, because different products and teams face different risks.

Note: One of the easiest traps is treating security as a late testing activity; the paper is strongly against that view.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 5

5. Application security controls: what are they?

- ▶ Note that SAFECODE uses application security controls (ASCs) interchangeably with **security requirements** (see footnote on the PDF).
- ▶ These controls are the actionable technical requirements that tell teams what must be protected and how.
- ▶ Good security requirements are:
 - clear,
 - actionable,
 - refined over time,
 - tied to threats, risks, and compliance needs.
- ▶ They support both engineering work and later auditing / measurement.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 7-8

6. Managing security controls as a workflow

- ▶ SAFECODE gives a simple workflow:
 1. identify threats (*what can hurt us?*), risks (*how bad it is if it happens?*), and compliance drivers (*what rules are we required to follow?*),
 2. derive security requirements,
 3. communicate them to the right teams,
 4. validate implementation,
 5. audit
- ▶ A notable recommendation: manage controls as **structured data in an ADLM¹ system**, not as loose text in an unstructured document.
- ▶ This makes security requirements trackable through implementation and verification.

Note: Operational traceability – being able to track security requirements from definition to implementation and validation – is essential

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 7-8

¹<https://www.gartner.com/reviews/market/application-development-life-cycle-management>

7. Design

- ▶ Secure software needs both **security features** and a **design that resists attacks**.
- ▶ The design stage is where teams decide authentication, authorization, auditing, encryption, and system boundaries.
- ▶ The document treats design flaws as especially dangerous because they can be catastrophic and hard to fix later.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 9-10

8. Secure design principles: the **classic**² set

- ▶ **Economy of mechanism:** keep the design as simple and small as possible.
- ▶ **Fail-safe defaults:** prefer permission by explicit allow, not vague openness.
- ▶ **Complete mediation:** every access to every object should be checked.
- ▶ **Least privilege:** users and programs should have only the privileges they need.
- ▶ **Least common mechanism:** minimize shared mechanisms among users.
- ▶ **Psychological acceptability:** security should be usable enough that people apply it correctly.
- ▶ **Compromise recording:** in some cases, reliable recording of compromise matters too.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), p. 9

²More information: <https://web.mit.edu/Saltzer/www/publications/protection/Basic.html>

9. Secure design principles: the added modern set

- ▶ **Defense in depth:** the system should still resist attack if one control fails.
- ▶ **Fail securely:** a crash or error should not turn into a security bypass.
- ▶ **Design for updating:** no system stays vulnerability-free forever, so updates must be planned from the start.
- ▶ SAFECODE stresses that these principles apply across product types: cloud services, clients, and IoT systems.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), p. 10

10. Threat modeling³

- ▶ Threat modeling is described as a security-focused design activity with one of the best returns on investment.
- ▶ Its strongest benefit comes **before code is committed**, when flaws are still cheap to change.
- ▶ Even later, threat modeling still helps by mapping the system, exposing weak spots, and focusing later testing.
- ▶ SAFECode does not insist on one perfect method; it insists on a **repeatable and manageable** process that identifies threats.

Source: SAFECode, Fundamental Practices for Secure Software Development (2018), p. 10

³https://safecode.org/wp-content/uploads/2017/05/SAFECode_TM_Whitepaper.pdf

11. Architectural review and encryption strategy

- ▶ Architectural and design review should check whether the design really follows the principles just introduced.
- ▶ Encryption should be considered during design, not bolted on later.
- ▶ SAFECode recommends that larger organizations centralize encryption strategy and governance through experts.
- ▶ Threat modeling helps decide **where encryption is needed**, but encryption strategy is still its own design task.

Note: The paper treats cryptography as too subtle to leave to ad hoc⁴ decisions by each feature team.

Source: SAFECode, Fundamental Practices for Secure Software Development (2018), pp. 10-11

⁴Ad hoc (Latin for “for this”): done in an improvised or unstructured way, typically for a specific situation rather than as part of a systematic process.

12. Encryption strategy: what must be decided (1/2)

- ▶ What data needs protection in transit and at rest⁵.
- ▶ Which mechanisms are allowed: vetted libraries, strong algorithms, strong protocol versions:
 - For example, all versions of SSL are considered broken, and early versions of TLS are no longer recommended due to concerns about weaknesses in the protocol.
- ▶ How keys and certificates are managed, rotated, and audited. Some related material next:
 - Recommendation for Key Management: Part 1 – General⁶
 - OWASP Key Management Cheat Sheet⁷
 - CloudSecurityAlliance⁸

Source: SAFECode, Fundamental Practices for Secure Software Development (2018), pp. 11-12

⁵At rest: data stored on disk or other persistent storage, not actively being transmitted or processed.

⁶<https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>

⁷https://cheatsheetseries.owasp.org/cheatsheets/Key_Management_Cheat_Sheet.html

⁸<https://cloudsecurityalliance.org/artifacts/key-management-lifecycle-best-practices>

12. Encryption strategy: what must be decided (2/2)

- ▶ Note that cryptographic assets (such as keys and certificates) must be assigned a bounded lifetime (**cryptoperiod**) and supported by a managed rotation process.
- ▶ Also note that **hard-coded secrets are unacceptable**.
- ▶ How to have **cryptographic agility**: the ability of a system to replace or upgrade cryptographic algorithms, keys, and libraries without major redesign or disruption.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 11-12

13. Identity and access management (IAM): standardize it

- ▶ Authentication and authorization are called **foundational controls** on which other controls depend.
- ▶ Organizations should standardize their approach so products are consistent and easier to verify.
- ▶ The document covers:
 - user authentication,
 - service-to-service authentication,
 - authorization of actions,
 - logging and auditing of security-relevant actions.
- ▶ Rebuilding authentication separately inside every feature team is framed as risky.

Source: SAFECode, Fundamental Practices for Secure Software Development (2018), pp. 12-13

14. Authorization: default deny, complete mediation, least privilege (1/3)

- ▶ Every access or action should be checked for authorization: this is complete mediation applied in practice.
- ▶ The safest structure is **default deny**: without an explicit authorization check, access should not happen.
- ▶ Authorization should also reflect **least privilege**: only the minimum permissions needed.
- ▶ SAFECODE mentions several possible authorization models (role-based, attribute-based, mandatory, discretionary).

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 13-14

14. Authorization: models (2/3)

Mandatory Access Control (MAC): access is enforced by the system based on fixed security rules; users cannot modify permissions (e.g., military or government systems with classified data).

Discretionary Access Control (DAC): access is controlled by resource owners, who can grant or revoke permissions (e.g., file systems, collaborative environments).

Mandatory and Discretionary Access Control (MAC/DAC) were defined in the [Trusted Computer System Evaluation Criteria](#)⁹ (1985) and are also reflected in [NIST SP 800-53](#)¹⁰ (2020).

⁹<https://csrc.nist.gov/files/pubs/conference/1998/10/08/proceedings-of-the-21st-nissc-1998/final/docs/early-cs-papers/dod85.pdf>

¹⁰<https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>

14. Authorization: models (3/3)

Role-Based Access Control (RBAC): access is assigned through roles, with users inheriting permissions based on their role (e.g., enterprise systems with well-defined job functions). See the NIST project page [Role-Based Access Control](https://csrc.nist.gov/Projects/role-based-access-control/publications)¹¹ for more information. Note that this project is no longer maintained; however, it provides useful historical context and a list of publications. You may also want to consult [INCITS 359-2012](https://blog.ansi.org/ansi/role-based-access-control-rbac-incits-359/)¹² for additional details.

Attribute-Based Access Control (ABAC): access decisions are based on attributes of the user, resource, and context (e.g., time, location) (e.g., dynamic or large-scale systems requiring fine-grained policies, such as cloud services). Check [NIST SP 800-162 - Guide to Attribute Based Access Control \(ABAC\) Definition and Considerations](https://csrc.nist.gov/pubs/sp/800/162/upd2/final)¹³ (last update 2019) for more information.

¹¹<https://csrc.nist.gov/Projects/role-based-access-control/publications>

¹²<https://blog.ansi.org/ansi/role-based-access-control-rbac-incits-359/>

¹³<https://csrc.nist.gov/pubs/sp/800/162/upd2/final>

15. Logging and audit: more than “turn on logs”

- ▶ Logs are essential for understanding incidents, investigations, and SIEM¹⁴ alerting.
- ▶ Logging design decisions depend on business needs, system design, threat model, and the teams that will use the logs.
- ▶ The paper stresses several tensions:
 - capture enough information,
 - avoid logging unnecessary sensitive data,
 - protect logs against tampering,
 - keep logs long enough for investigation.
- ▶ Runtime configurability of logging is recommended.

Note: Security controls must be useful in operations, not merely present in code.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), p. 14

¹⁴<https://www.microsoft.com/en-us/security/business/security-101/what-is-siem>

16. Secure coding: the main idea

- ▶ Developers make mistakes; the SDL should reduce the number of mistakes that become vulnerabilities.
- ▶ SAFECODE proposes several levers:
 - coding standards,
 - safe language / library / framework / compiler choices,
 - tool support,
 - safe data handling (“all user-originated input should be treated as untrusted”),
 - robust error handling,
 - careful use of third-party components.
- ▶ The section is not just “write better code”; it is about shaping the environment in which code is written.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 15-16

17. Secure coding: coding standards

- ▶ Once technology choices are made, teams should define coding standards and conventions that support secure use of those technologies.
- ▶ Built-in security features of frameworks and tools should be used whenever possible, ideally enabled by default.
- ▶ SAFECODE explicitly advises against re-inventing security features in-house when mature framework support already exists.
- ▶ Standards should also be realistic and enforceable, with validation in mind from the beginning.

Note: Reuse secure components, but do so deliberately and with verification.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), p. 15

18. Secure coding: safe functions, updated toolchains, and secure compiler options

- ▶ Developers should know which functions and APIs are dangerous and what the safer alternatives are.
- ▶ Tooling should help identify banned or risky function use during development or builds.
- ▶ SAFECode also stresses current compiler / toolchain versions and secure compiler options.
- ▶ The point is not cosmetic modernization: newer toolchains and mitigations can reduce exploitability of common bug classes.

Note: It is recommended to use safe defaults from both source code choices and build-time protections.

Source: SAFECode, Fundamental Practices for Secure Software Development (2018), pp. 15-17

19. Secure coding: code analysis tools: what role do they play?

- ▶ Tools help find deviations from requirements and coding guidance early in the build cycle.
- ▶ They should be integrated into the normal development workflow rather than used as a special extra step.
- ▶ SAFECODE also warns that frameworks only protect up to a point: developer-created logic bugs can still create vulnerabilities.
- ▶ So analysis tools help, but do not replace secure review and later testing.

Note: Automation is crucial but not sufficient by itself.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), p. 17

20. Secure coding: handle data safely 1/2: input is untrusted

- ▶ SAFECODE calls this “Security 101”: all user-originated input should be treated as untrusted.
- ▶ Data can come from many channels: networks, files, IPC, or other applications.
- ▶ The paper rejects the simplistic idea that validation only happens at the internet boundary.
- ▶ Better model: **each component protects itself** against malicious input relevant to it.
- ▶ Parameterized queries are given as an example of defending a component against SQL injection.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 17-18

21. Secure coding: handle data safely 2/2: canonicalize, validate, sanitize

The paper distinguishes several ideas that people often blur together:

- ▶ **Canonicalization**: reduce multiple equivalent representations to one standard form – think about multiple forms of writing a date.
- ▶ **Sanitization**: modify or clean data so a component can consume it safely – think about removing any strange artifact from, for example, a date: `10-01-2026; exec ls`; would be sanitized into `10-01-2026`.
- ▶ **Validation**: decide whether input is acceptable or reject it – for example, reject invalid dates such as `31/02/2026` or `<script>alert(1)</script>` or `10-01-2026; exec ls`;

Validation is often safer than “cleaning up” (sanitization) bad input after the fact¹⁵.

Source: *SAFECode, Fundamental Practices for Secure Software Development* (2018), pp. 18-19

¹⁵After the fact: done after an event has already occurred, rather than beforehand.

22. Secure coding: handle errors securely

- ▶ Applications must react to unanticipated errors in a controlled and graceful way.
- ▶ Generic handlers are needed for unexpected failures.
- ▶ If a generic handler is triggered, the current action should stop, because the system may be in an unknown state.
- ▶ User-facing errors should be generic; detailed technical information belongs in logs for administrators, not in attacker-visible messages.

Note: This section connects fail securely, logging, and usability. You should protect the user experience without helping the attacker by revealing internal information.

Source: *SAFECode, Fundamental Practices for Secure Software Development* (2018), p. 20

Third-party components: reuse and distrust at the same time

- ▶ Developers increasingly depend on frameworks, libraries, open source software (OSS), and commercial off-the-shelf-products (COTS).
- ▶ SAFECode clearly supports using established components instead of re-implementing everything.
- ▶ But it equally stresses that these components bring **inherited risk**.
- ▶ Treating them as black boxes with less scrutiny than in-house code is **dangerous**.
- ▶ The key idea: reuse is sensible, blind trust is not.

Source: SAFECode, Fundamental Practices for Secure Software Development (2018), p. 21

24. Testing and validation: why multiple methods are needed

- ▶ Security testing is often the first SDL activity organizations adopt.
- ▶ In mature programs, testing has two roles:
 - validate that security practices are working,
 - catch flaws that still got introduced.
- ▶ SAFECODE divides testing into **automated** and **manual** approaches and recommends to use several kinds, not one.

Note: The question is not “which testing method is best?” but “what does each method contribute?”

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 22-24

25. Automated testing: SAST and DAST

SAST (Static Analysis Security Testing)

- ▶ Inspects source/intermediate/binary code.
- ▶ Good coverage and good fit for build automation.
- ▶ Limited on some business-logic and runtime issues.

DAST (Dynamic Analysis Security Testing)

- ▶ Tests a running application.
- ▶ Can test integrated behavior and actual exploitability.
- ▶ Slower and depends on what functionality it can discover or drive.

SAFECode presents them as complementary, not competing.

Note: Intuitively, SAST sees structure and DAST sees behavior.

Source: SAFECode, Fundamental Practices for Secure Software Development (2018), pp. 22-23

26. Other automated testing categories

- ▶ **Fuzzing**: especially valuable for parsers, and especially for code in non-memory-safe languages.
- ▶ **Network vulnerability scanning**: useful for installed environments and appliances, especially for known issues.
- ▶ **Configuration / mitigation verification**: check use of platform mitigations, secure headers, TLS configuration, cookie flags, etc.
- ▶ **Automated functional testing of security features**: test security behavior just like any other feature.

Note: A nice insight in this section: sometimes the easiest automation is not finding bugs directly, but checking that important mitigations are actually enabled.

Source: SAFECode, Fundamental Practices for Secure Software Development (2018), pp. 23-24

27. Manual testing and penetration testing

- ▶ Manual testing is slower and more expensive, but humans can adapt, infer, and learn.
- ▶ Manual verification should include security features and mitigations, not just functionality.
- ▶ Penetration testing is especially good at finding complex issues, including business-logic flaws, in realistic context.
- ▶ Because it is resource-intensive, SAFECODE suggests using it where it matters most, often after cheaper automated methods have already run.

Note: The paper values human expertise, but wants to spend it where it adds the most unique value.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 24-26

28. Manage security findings: discovery is not enough

- ▶ Findings from threat modeling, testing, review, and component analysis must be tracked.
- ▶ The organization must then choose to:
 - remediate,
 - mitigate,
 - or accept residual risk.
- ▶ SAFECODE recommends clear severity definitions and a tracking system that can report unresolved and resolved security issues.
- ▶ Consistent severity language matters so teams discuss risk in the same way.

Note: The SDL is not measured by how many issues it finds, but by how responsibly it handles them.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 27-28

29. Risk acceptance

- ▶ Risk acceptance is allowed, but it should be structured, explicit, and approved by the right level of authority.
- ▶ SAFECODE recommends a defined process and archived records.
- ▶ Acceptance records should include severity, residual risk, mitigation details, verification, and a plan for re-review or expiration.
- ▶ In other words, accepted risk should not become a **forgotten risk**.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), p. 28

30. Vulnerability response and disclosure

- ▶ SAFECODE assumes vulnerabilities may still exist in released software.
- ▶ Therefore organizations need:
 - internal and external response policies,
 - defined roles and responsibilities,
 - clear contact channels for reporters,
 - timely acknowledgement and coordination,
 - a process to fix, mitigate, and disclose responsibly.
- ▶ The section also references ISO guidance and the role of PSIRT¹⁶-like functions.

Note: Post-release security is not outside the SDL; it is a core part of it.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 29-32

¹⁶Product Security Incident Response Teams

31. Third-party vulnerabilities and feedback into the SDL

- ▶ Organizations must monitor vulnerabilities discovered in third-party components before and after releasing a product.
- ▶ Fixing a reported issue should also trigger a broader question: does the same defect class appear elsewhere?
- ▶ It is recommended to perform a root-cause analysis and using what is learned to improve the SDL.
- ▶ This feedback loop is central to continuous improvement.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 30-32

32. Planning and rolling out an SDL inside an organization

- ▶ The final section moves from technical practices to program design.
- ▶ SAFECODE asks organizations to think about:
 - culture,
 - expertise and training,
 - product lifecycle model,
 - scope of rollout,
 - stakeholder¹⁷ communication,
 - compliance measurement,
 - process health,
 - value proposition.

Source: SAFECODE, Fundamental Practices for Secure Software Development (2018), pp. 33-36

¹⁷Stakeholder: any person or organization that can affect, be affected by, or has an interest in the outcome of a system or project.

33. Key Takeaways

- ▶ Security is **not a step** but a **continuous lifecycle process**.
- ▶ Strong security starts with **design**: threat modeling and secure principles.
- ▶ **Implementation choices matter**: safe APIs, data handling, and dependency management.
- ▶ **No single testing method is sufficient**: combine SAST, DAST, fuzzing, and manual testing.
- ▶ **Findings must be managed**: prioritize, remediate, or explicitly accept risk.
- ▶ **Release is not the end**: vulnerability response and feedback improve future security.
- ▶ Ultimately: security is about **process, discipline, and continuous improvement**.